

Building Faster Transformer with Triton

A hands-on introduction to GPU programming



Why do Transformers Need Fast Custom Kernels?

Transformers are mostly repeated tensor operations

$$\mathbf{Q} = \mathbf{X} @ \mathbf{W}_q$$

$$\mathbf{K} = \mathbf{X} @ \mathbf{W}_k$$

$$\mathbf{V} = \mathbf{X} @ \mathbf{W}_v$$

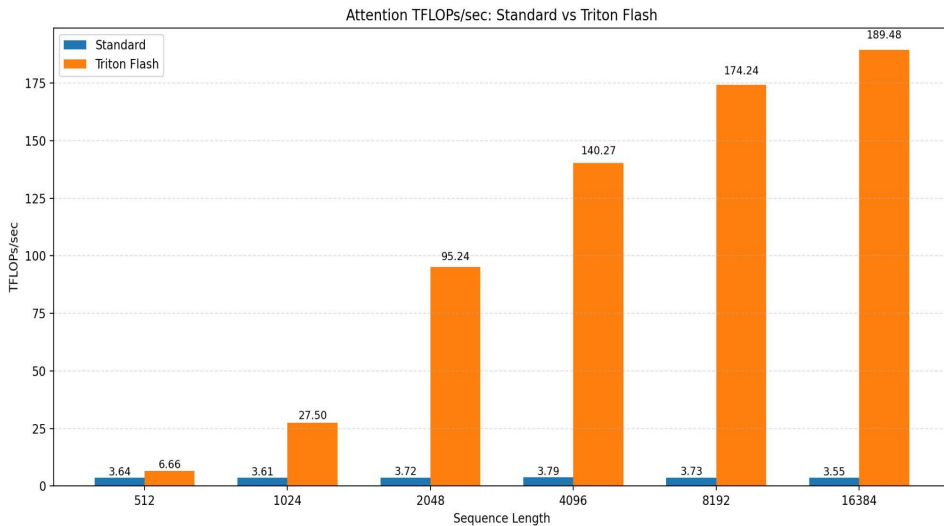
$$\text{scores} = \mathbf{Q} @ \mathbf{K}^T$$

$$\text{output} = \text{softmax}(\text{scores}) @ \mathbf{V}$$

These operations run millions or billions of times during training and inference. If these operations are slow, the model is slow.

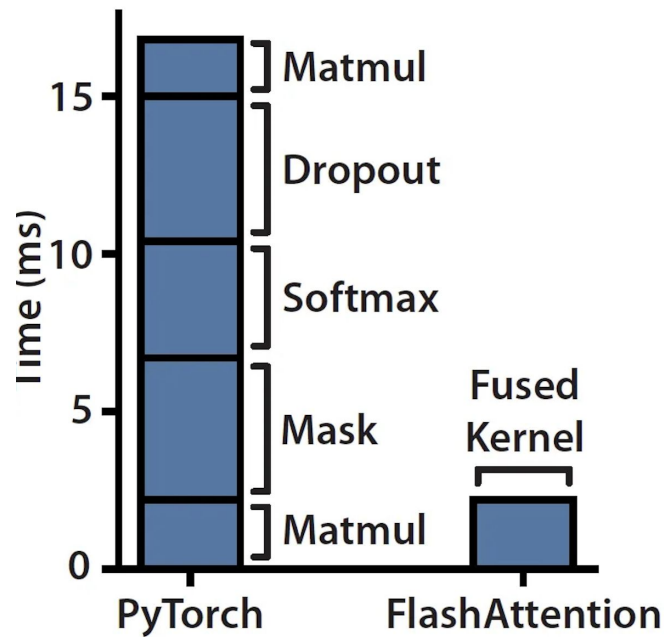
But why can two implementations that perform the same mathematical operation have very different runtimes?

Same Maths, Different Runtime

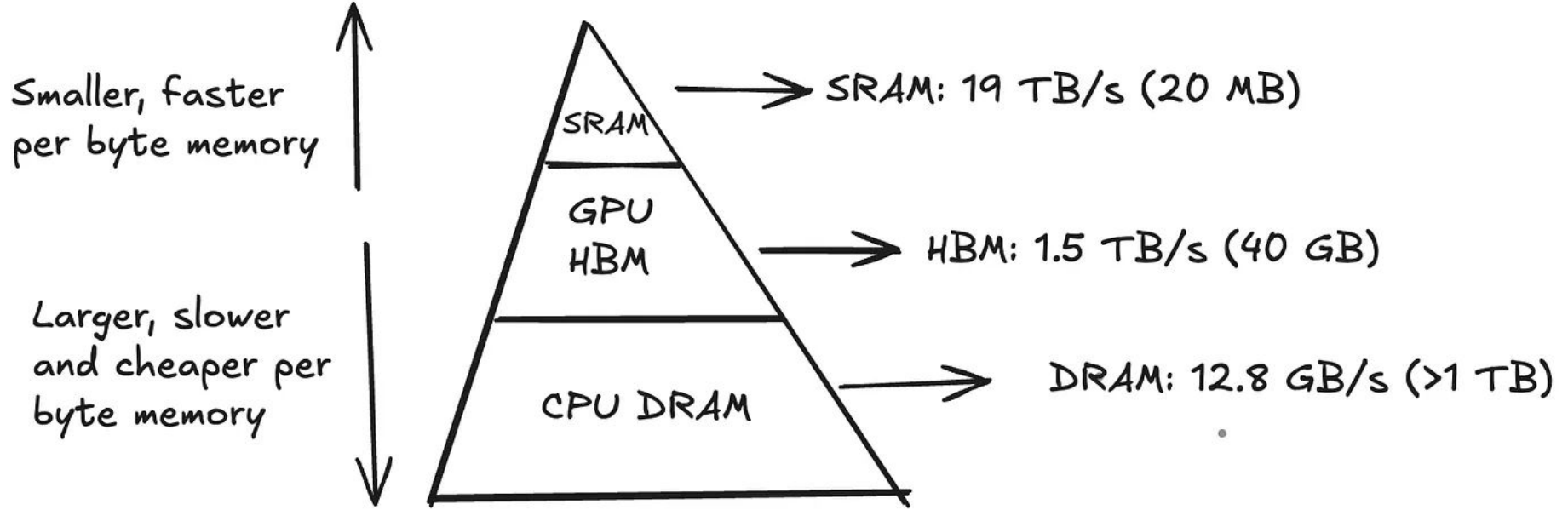


Both implementations produce the same **output** matrix

But performance depends on:



- how the work is divided
- how often data is loaded
- where the data is stored
- whether loaded data is reused
- how well the GPU is kept busy



The Core Problem: Memory Movement

Modern GPUs can
do arithmetic very
quickly

**But before
arithmetic can
happen...**

Load data from the
global memory
→ Compute → Store
result

If we keep loading the same data from global memory again and again,
the arithmetic units may sit idle.

The number that actually matters: arithmetic intensity

A GPU's spec sheet brags about FLOPS (compute). But most kernels are not limited by compute — they're limited by how fast you can move data from memory to the chip.

Arithmetic intensity = FLOPs performed / bytes moved from memory.

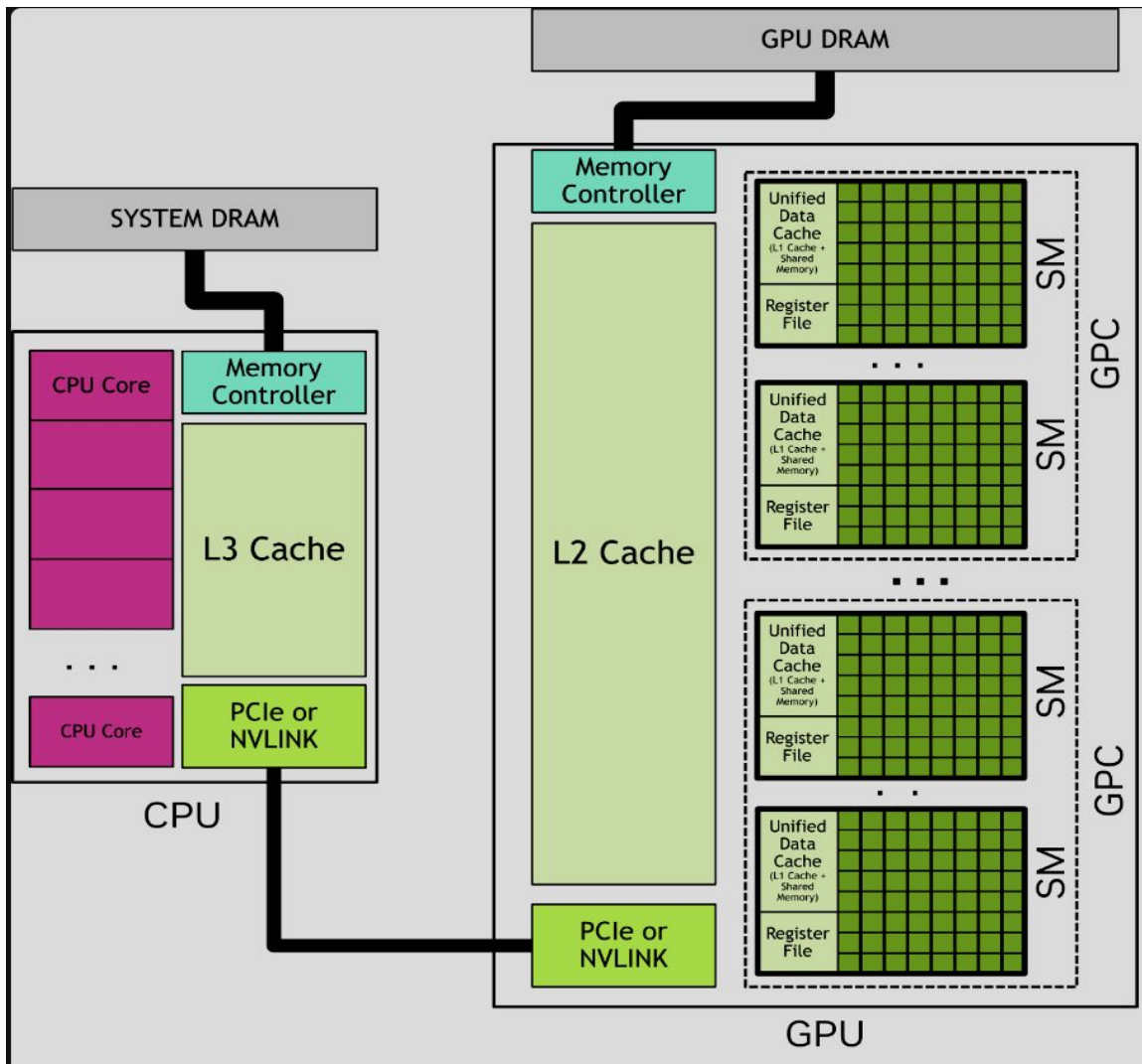
For an $N \times N$ matmul, naively:

- FLOPs: $2 * N^3$ (one multiply + one add per output, accumulated N times)
- Bytes moved (naive, no reuse): each output element re-reads a full row and column $\Rightarrow O(N^3)$ byte traffic too

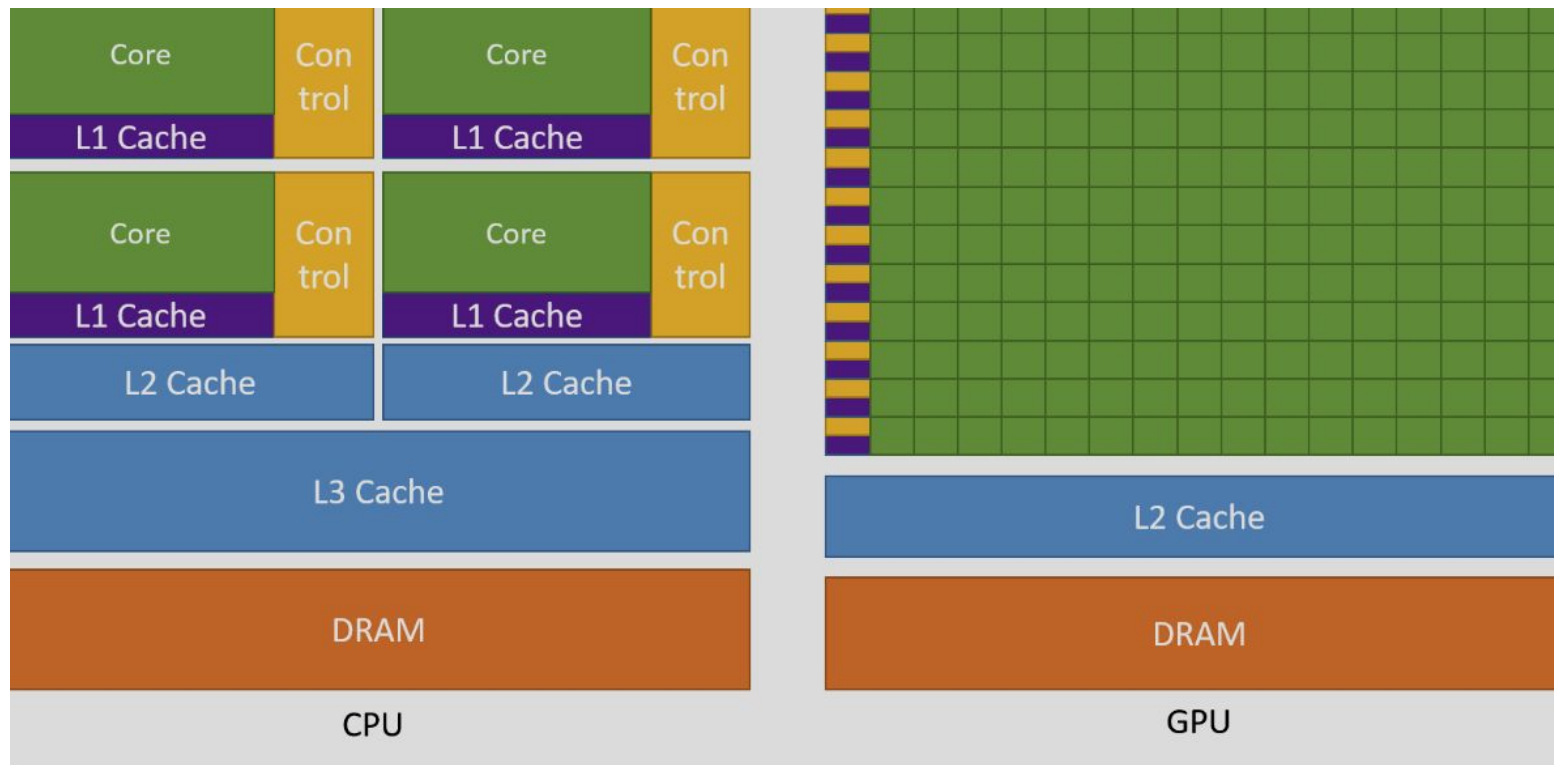
This is the whole reason tiling exists: cut down on how many times bytes get pulled from slow global memory before useful compute happens on them.

A GPU can have thousands of cores and still be slow, because those cores spend most of their time waiting on memory, not computing. Tiling fixes exactly this number it raises arithmetic intensity by reusing data that's already on-chip instead of re-fetching it.

Fast kernels reduce unnecessary memory movement.



The CPU vs. GPU Mental Model



The GPU dedicates more transistors to data processing.

GPU Mental Model: GPU work is split into many pieces

A GPU kernel is not one computation.

It is one program replicated across many independent pieces of data.

Same kernel code runs across many thread blocks or program instances over different regions of the input tensor

For a vector:

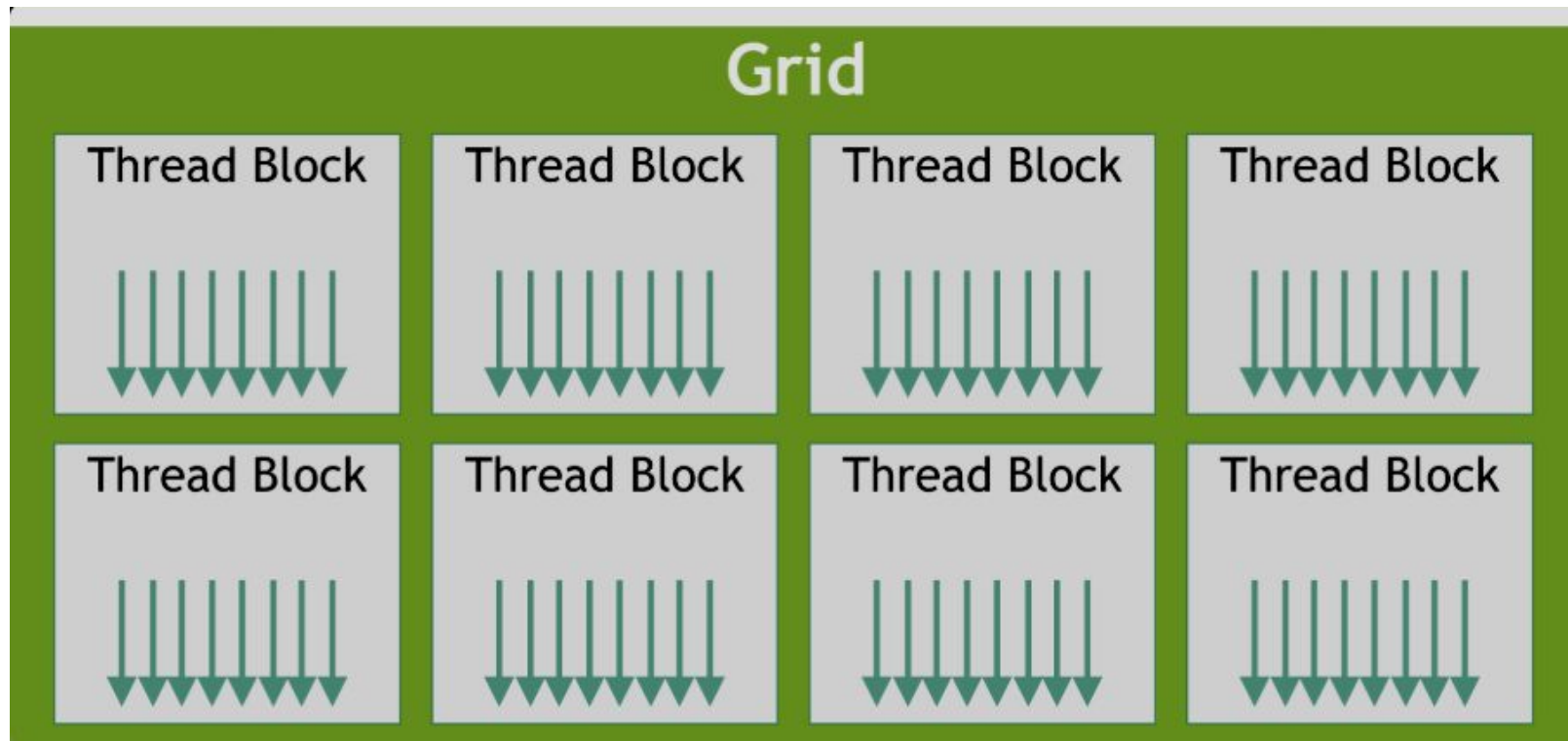
[0 1 2 3 | 4 5 6 7 | 8 9 10 11 | 12 13 14 15]
block 0 block 1 block 2 block 3

Each block handles a part of the data.

In Triton, we usually think in terms of:

program ID → offsets → load → compute → store

Triton gives us a block-level view. Instead of thinking about one scalar thread at a time, we think about one program handling a block of elements.



Grids, Blocks, Tiles and Threads

CUDA View

Many threads
execute the same
kernel.
Each thread often
handles one
element.

Triton View

Many programs
execute the same
kernel.
Each program
often handles a
block/tile of
elements.

Now let's write our first Triton kernel.

Your First Triton Kernel

A Vector Addition Kernel in Triton

The triton programming pattern you will see throughout:

program ID $\rightarrow$ offsets $\rightarrow$ mask

Load $\rightarrow$ Compute $\rightarrow$ Store

A Triton kernel is executed by many **program instances**. Each program uses its ID to determine which part of the tensor it owns.

The vector-add kernel follows six steps:

1. Get the program ID
2. Calculate this program's offsets
3. Create a boundary mask
4. Load x and y
5. Add them
6. Store the result

Start by launching a grid. A grid is the collection of all Triton program instances launched for one kernel call. Specifies how many program instances to launch.

Each Triton program handles a block of elements. A program ID identifies a Triton program instance, which normally maps to a GPU thread block. That program instance operates on one tile of data, represented inside the kernel using vectors of offsets and values. The program ID tells us which block. Offsets identify the elements, and the mask protects the final incomplete block.

A `@triton.jit` function describes what each/one program instance does.

The grid dimensions tells Triton/the GPU how many copies of that instruction to run across the GPU SM. Each program is pointed/offset to a part of the data (tensor in our case) to work on and where to accumulate/write its result to.

Why do we need a mask?

The final block
may extend past the
end of the tensor.

Tensor size = 10

Block size = 4

Program 0: 0, 1, 2, 3

Program 1: 4, 5, 6, 7

Program 2: 8, 9, ?, ?

mask = offsets < n_elements

Hands-on Exercise: Vector Addition

1. Complete offsets
2. Create the mask
3. Run the kernel
4. Check correctness
5. Benchmark against PyTorch

Goal: Focus on how a program maps to a block of data. Don't memory the syntax,.

Break

Why Matrix Multiplication Needs Tiling

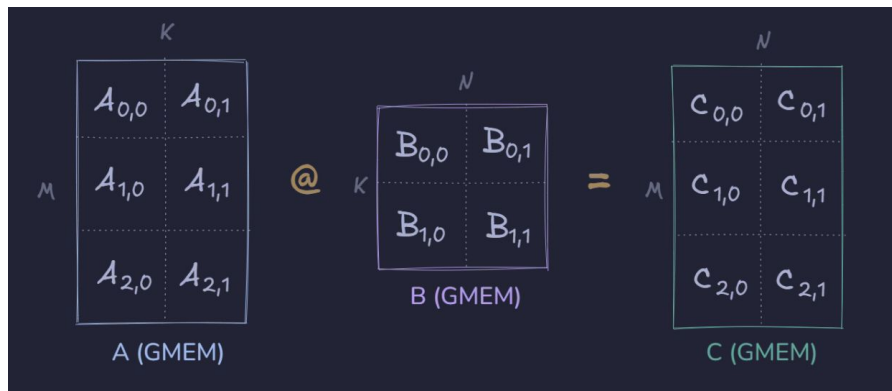
Matrix multiplication is particularly well-suited for GPU acceleration due to the independent nature of computing each output element.

$$C[i, j] = \sum A[i, k] \times B[k, j]$$

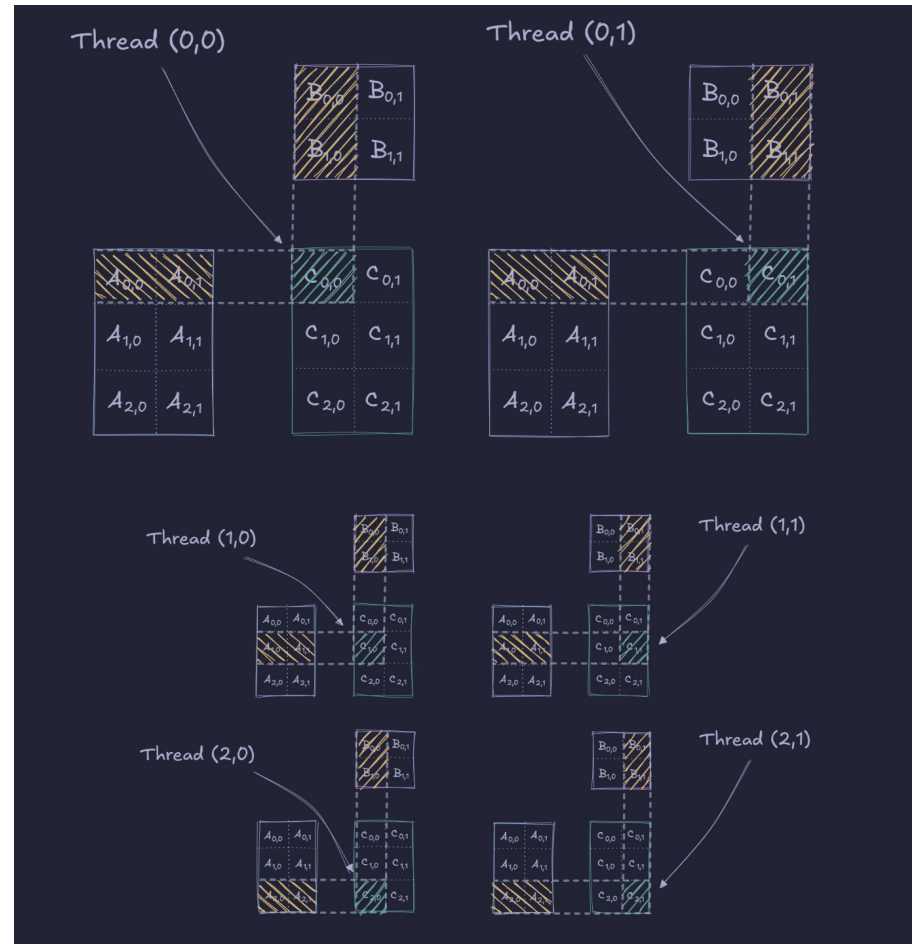
One output element uses one row of A and one column of B.

The naive strategy of matrix multiply repeatedly loads the same data

When neighbouring output elements are computed independently, the same rows and columns may be fetched from global memory again and again.



Assigning each output element to a dedicated CUDA thread. Thus, each thread in this implementation needs to:



Tiling changes the question

Instead of:

How do we compute one output value?

Ask:

How do we compute one output tile?

Load a tile of A and a tile of B once, reuse them for many multiply-add operations, and accumulate a tile of C.

The tiling strategy:

1. Each block loads a small tile of A and a small tile of B into shared memory.
2. All threads in the block compute partial sums using ONLY that shared tile.
3. Sync threads, slide the tile over, repeat until the full row/column has been covered.

This turns "N redundant reads from global memory" into "1 read from global memory, N reuses from shared memory."

One Triton program
computes one output
tile.

Reading the tiled matrix-multiplication kernel

- 1. Choose an output tile**
- 2. Load one A tile and one B tile**
- 3. Multiply and accumulate**
- 4. Move along K**
- 5. Store the output tile**

The kernel repeats the same tiled computation across the output matrix in parallel.

Hands-on exercise: change the tile shape

Change one thing
at a time, then
measure.

Try:

$16 \times 16 \times 16$

$32 \times 32 \times 16$

$32 \times 32 \times 32$

$64 \times 32 \times 32$

For each configuration:

check correctness

measure runtime

compare with `torch.matmul`

Five ideas to remember

Program IDs choose the work.
Offsets choose memory locations.
Masks protect tensor boundaries.
Tiling improves data reuse.
Correctness comes before performance